

TP 03

Programación II

Contenidos
Implementación del TDA List

Consigna:

- Dentro del package nombre **listModule**, crear una interfaz **SimpleList<E>**, y las clases **SimpleArrayList<E>** y **SimpleLinkedList<E>**.
- Reemplazar en ListExercise las variables con Listas de Java por las creadas para este TP.
- Para referencia, pueden utilizar la [documentación de Java sobre List<E>](#).
- SimpleList<E>**:
 - Métodos:**
 - public boolean add(E element).
 - Inserta element al final de la lista, devuelve true siempre.
 - public void add(int index E element).
 - Inserta element en el index dado, corriendo todos los elementos en ese index y los siguientes a la derecha / un índice más arriba.
 - public E remove(int index)
 - Remueve el elemento de la lista en el index dado, y lo devuelve.
 - public boolean remove (Object object)
 - Remueve object pasado por referencia, devuelve true si se encontró y pudo ser removido de la lista.
 - public void clear().
 - Borra todos los elementos de la lista, dejándola vacía.
 - public boolean contains(Object object).
 - Devuelve true si la lista contiene el object pasado por referencia.
 - public E get(int index).
 - Devuelve el elemento ubicado en index.
 - public E set(int index, E element).
 - Reemplaza el elemento ubicado en index por element, y devuelve el elemento que estaba ahí antes de ser reemplazado.
 - public int size().
 - Devuelve la cantidad de elementos en la lista.
 - public boolean isEmpty().
- SimpleArrayList<E>**:
 - Implementación estática de SimpleList<E>.
 - Internamente trabaja con un array; si ese array se llena debe ser reemplazado por uno nuevo con el doble de tamaño, y todos los elementos que ya estaban guardados se copian.
 - size cuenta sólo los espacios ocupados. Si el array tiene 8 espacios y 3 en uso, size = 3.
 - No deben quedar espacios vacíos entre elementos. Todos los elementos deben tener índices contiguos (uno al lado del otro), y si hay espacios vacíos deben estar todos al final.
 - Al insertar o remover en el medio, se deben correr los elementos hacia adelante o atrás.
- SimpleLinkedList<E>**:
 - Implementación dinámica de SimpleList<E>.
 - Internamente trabaja con nodos enlazados. Cada nodo guarda un elemento genérico, más una referencia a los nodos anterior y siguiente, si existen. Si no existen se guarda null.
 - Cada vez que se inserta un nuevo nodo, se deben arreglar las conexiones:
 - Si se inserta al principio, su nodo siguiente es el que antes era primero, y el nodo previo del que antes era primero es el nodo insertado.
 - Si se inserta al final, su nodo previo es el que antes era el último, y el nodo siguiente del que antes era último es el nodo insertado.
 - Si se inserta entre dos nodos, consideremos nodo A al primero, B al segundo y C al que se quiere insertar:
 - Previo de C = A, Siguiente de nodo C = B
 - Previo de nodo B = C, Siguiente de nodo A = C